

Kubernetes

Cluster - instance of Kubernetes. Each cluster has control plane and at least one worker node.

Kubelet - An agent that runs on every worker node. Make sure that containers in a pod are running and healthy. It communicates directly with the api-server in the control-plane.

Container Runtime - Once the Kubelet has been assigned to a new Pod it starts the container using the Container Runtime Interface, CRI.

Common commands

`minikube start` - Start a minikube cluster.

`minikube delete` - Delete a minikube cluster.

`kubectl cluster-info` - Give you information about the cluster. Like IP and port number of the host.

`kubectl get nodes` - Give information about the kubernetes nodes.

`kubectl get pods -A` - Give us info about the pods where the kubernetes cluster is running.

`kubectl get services -A` - Give us info about the services which give us load balancers and direct traffic to pods.

`kubectl api-resources` - Give all the resources available to use.

`kubectl get pods -n kube-system` - How you can see Kube API server which is a containrized application.

yaml file

Kubernetes runs as Infrastructure as Code - so by run **apply** command on yaml file you can create different entities.

Entity we are familiar with - Namespace, Deployment, Pod, Service. The **kind:** **<entity>** define which entity to create.

Namespace

Namespace let you isolate and organize your applications and microservices.

`kubectl get namespaces` - Give info about the namespaces created by the cluster.

`kubectl apply -f namespace.yaml` - Create new namespace from a yaml file.

`kubectl delete -f namespace.yaml` - Remove namespaces defined inside certain yaml file.

Deployment

Under some namespace a deployment entity lives, every deployment has pods which are basically the applications we run defined by container image file.

We can define how many “replicas”, copies, we want for every application, each copy lives in different pod. When one pod is terminated a new one will be create to maintain the number of copies we defined.

`kubectl apply -f deployment.yaml` - Create new deployment

`kubectl get deployments -n development` - Get all the deployments in the “development” namespace.

Pods

Pods are the Kubernetes resource that run applications and microservices.

`kubectl get pods -n development -o wide` - Get all the pods with wide info in the “development” namespace.

`kubectl describe pod <pod-label> <pod-name> -n <pod-namespace>` - Get all the logs and info related to a specific pod.

`kubectl logs <pod-label> <pod-name> -n <pod-namespace>` - Get all the logs.

BusyBox, exec, quote

A tool let you check on an application. good for debugging in linux. As any application you will create him as an application runs inside a pod from an image file called “busybox:latest”.

`kubectl apply -f busybox.yaml`

`kubectl exec -it <pod-name> -- /bin/sh` - Get inside the pod (containter) in interactive mode.

`wget <pod-ip>:<pod-port>` - busybox application gives us information about other pods. He uses the IP and port that defined in the other pod yaml.

So if we create “quote” pod which is a simple application uses the image file datawire/quote:0.5.0 and defined in the `quote.yaml` file that gives us some random quotes. We can check the application data with busybox.

Kubernetes LoadBalancer Service

Kubernetes Service with a type LoadBalancer is how we expose a an application to the server. A load balancer that direct traffic from the Internet to Kubernetes Pods.

A load balancer service as such has a public IP and static IP address.

The Public IP address means that anyone can access it from the Internet. And the static IP address is important because the pods and their IP address are always changing but service IP must stay the same.

We must define port for the input traffic and targetPort where the load balancer will send traffic for our pods.

`minikube tunnel` - Create a connection to the Internet, if you run it locally on your computer than you don't actually get an IP and the result will be 127.0.0.1.

`kubectl apply -f service.yaml` - Create the LoadBalancer Service.

`kubectl get services -n deployment` - Find the external IP address we expose to the public internet.

Process of deployment

- 1) Command: Command `kubectl apply -f namespace.yaml` - Our kube-config file gives permission to kubectl to communicate with our kubernetes cluster.
- 2) Save new state: The kubectl command sends the information to the Kube API Server which saves the new deployment spec in etcd.
- 3) Check for changes: The kubectl manager is checking the Kube API Server to see if there have been any changes since its last loop.
- 4) Watch for unassigned pods: Since there is new deployment the Kube Scheduler checks the Kube API Server to see if there is any new Pods that have not been assigned a node
- 5) Notify about Pod with Node name: The Kube API Server tells the scheduler that indeed there is a new Pod that has not been placed on a specific node.
- 6) Assign Pod to Node: So the scheduler chooses a node for the pod and sends that information back to the Kube API Server.
- 7) Saves new State: The API Server saves the state of the cluster in etcd.
- 8) Look for newly unassigned pods: In the worker node - The Kubelet check the API Server to see if there is any new pods that has been assigned.
- 9) Bind Pod to Node: The Kube API Server sends the pod spec for the new pod to the kubelet.
- 10) Start Container: The Kubelet pulls the image and create the container using the Container Runtime.
- 11) Update Pod status: Once the pod has been created the Kubelet sends the Pod status, healthy or not healthy back to the API server.
- 12) Save new state: The API Server update the state to the etcd.

Kubernetes Cluster Component

A Comprehensive Glossary of Kubernetes Cluster Components

Control Plane 1) Cloud Controller Manager: Connects a Kubernetes cluster to a cloud provider's API, managing cloud-specific resources and ensuring proper integration with the underlying infrastructure

2) etcd: A key-value store that saves all data about the state of the cluster; only the kube

3) kube-apiserver: The kube-apiserver is a key component of Kubernetes that exposes the Kube

4) kube-controller-manager: Monitors the Kubernetes cluster's state, running processes to en

5) kube-scheduler: Identifies a newly created pod that has not been assigned a worker node a

Worker Nodes 1) Container Runtime: Pulls container images, creates and manages containers, and ensures they run properly and securely as directed by the Kubernetes control plane

2) kube-proxy: A network proxy that runs on each node in a Kubernetes cluster, maintaining n

3) kubelet: An agent that runs on each node in a Kubernetes cluster, ensuring containers in